

Deck of 52 Cards

Object-oriented design, Fisher-Yates shuffle, and fair dealing

A classic low-level OOP interview. Interviewers want a clean model, an unbiased shuffle you can prove, and a deal that does not starve players.

How to use this document in an interview: start with the 90-second pitch, draw the class diagram, name the data structures and their complexities, then walk the core flow. Use the SOLID and follow-up sections when the interviewer pushes on extensibility.

1. 90-second pitch

I model the domain as four types: Suit and Rank are closed enums, Card is an immutable pair of those enums, and Deck owns a list of 52 unique cards. The top of the deck is the end of the list so a draw is $O(1)$. Shuffle is in-place Fisher-Yates: for i from $n-1$ down to 1, swap i with a uniformly chosen j in $0..i$. That produces each of the $52!$ permutations with equal probability. Deal is either `draw_card()` for one card or `deal_hands()` which walks the table round-robin so every player gets the same count and any positional bias is spread evenly.

SAY THIS: I would not shuffle with `random.shuffle` in an interview without saying that it is Fisher-Yates underneath. Implement it yourself so you can discuss uniformity and off-by-one bugs.

2. Problem statement and requirements

Functional

- Represent a standard 52-card deck (13 ranks x 4 suits, no jokers unless asked).
- Shuffle the deck so every permutation is equally likely.
- Draw / deal one or more cards from the top.
- Deal N cards to P players fairly.
- Reset the deck to a full unshuffled pack.
- Optionally show remaining cards and player hands.

Non-functional / interview quality bar

- Correctness of uniqueness: no duplicate cards after construct or shuffle.

- Unbiased randomization (do not use swap-with-rand(n) for every index).
- Clear ownership: Deck mutates itself; Card does not.
- Fail loudly on over-deal rather than returning None or wrapping around.
- O(1) draw, O(n) shuffle, O(1) extra memory.

Assumptions I would state out loud

- Ace is low for display sort (value 1). If this is blackjack or poker, Ace may be high or dual.
- No jokers, no multiple decks, no cut card, unless the interviewer adds them.
- Players do not steal cards from each other; they only receive from the Deck.
- Fair deal means equal count, not equal strength of hands.

3. High-level design

Class diagram (what to draw on the whiteboard)



Composition, not inheritance: a Deck has Cards; a Card has a Suit and a Rank. There is no 'RedCard' subclass. Inheritance would be the wrong tool here.

Module map in this repo

Topic	What to say / what we did
cards/suit.py	Four suits with a print symbol and a label.
cards/rank.py	Thirteen ranks. numeric is sort strength; value is reserved by Enum.
cards/card.py	Immutable value object. Comparable and hashable.
cards/deck.py	52-card owner. Shuffle, draw, deal, reset.
cards/player.py	Named hand. Receives cards; does not draw from the deck itself.
cards_main.py	Console UI. No domain rules live here.

4. Class responsibilities

Suit and Rank

Enums are the right type because the set is closed. You cannot construct a fifth suit. Each member carries display data (label/symbol) plus, for Rank, a numeric order. Rank.numeric is named that way on purpose: Enum already owns .value (the raw tuple). Mentioning this shows you have hit the Python enum footgun.

Card

A Card is a value object. Identity is the pair (rank, suit). After construction it cannot change -- slots plus read-only properties. That lets us put cards in a set to prove uniqueness, and sort a hand without mutating it. @total_ordering fills in >, <=, >= from __eq__ and __lt__.

```
@total_ordering
class Card:
    __slots__ = ('_rank', '_suit')
    def __eq__(self, other):
        return (self._rank, self._suit) == (other._rank, other._suit)
    def __lt__(self, other):
        return (self._rank.numeric, self._suit.name) < (...)
    def __hash__(self):
        return hash((self._rank, self._suit))
```

Deck

Deck is the aggregate root. It is the only type that may reorder or remove cards from the pack. Internal list _cards: index 0 is the bottom, the last index is the top. Construction is a Cartesian product: for each Suit, for each Rank, emit one Card. That is $4 * 13 = 52$ and is automatically unique.

Player

Player is a separate aggregate so dealing is not 'return a list of lists'. The hand property returns a copy, so a caller cannot mutate the real hand. receive() is the only write path. This is Law of Demeter: Deck talks to Player, not to player._hand.

5. Data structures

Topic	What to say / what we did
List[Card]	Primary store. Contiguous, ordered, O(1) pop from end.
set(Card)	Used in tests / as a talking point to prove 52 unique hashes.
Enum	Closed vocabulary. Iteration order is declaration order.
Why not a queue?	collections.deque is also O(1) pop. A list with top-at-end is enough and simpler to shuffle in place.
Why not a linked list?	Shuffle needs random access. Linked list makes Fisher-Yates $O(n^2)$.

Interviewers sometimes ask 'how would you implement an infinite shoe' (blackjack). Answer: keep a factory that can reset/shuffle a new 52-card block, or hold several decks in one list ($6 * 52$) and shuffle once. The Card type does not change.

6. Key algorithm: Fisher-Yates shuffle

This is the part they will linger on. The modern (Knuth) Fisher-Yates shuffle:

```
def shuffle(self) -> None:
    cards = self._cards
    for i in range(len(cards) - 1, 0, -1):
        j = secrets.randbelow(i + 1) # uniform in 0..i inclusive
        cards[i], cards[j] = cards[j], cards[i]
```

Why this is unbiased

At step i , the card that lands in position i is chosen uniformly from the first $i+1$ cards (indices $0..i$). Those $i+1$ cards are exactly the ones not yet finalized. By induction, every permutation has probability $1/n * 1/(n-1) * ... * 1/1 = 1/n!$.

The naive bug they expect you to name

The broken version is: `for i in 0..n-1: swap(i, randrange(n))`. That is NOT uniform. Each swap has n choices, so you generate n^n outcomes mapped onto $n!$ permutations. n^n is not divisible by $n!$ for $n > 2$, so some permutations get more mappings than others. Example they love: $n=3$, 27 outcomes into 6 permutations -- cannot be equal.

WATCH OUT: Off-by-one: `randbelow(i)` instead of `randbelow(i+1)` excludes swapping with self and skews the distribution. Inclusive upper bound is required.

Why `secrets.randbelow`, not `random.randint`

`random` is a PRNG (Mersenne Twister). Fine for games. `secrets` is CSPRNG, uniform over the range, and is the right story if the interviewer says 'this is an online casino'. In a board-game app, `random.Random` with an optional seed is better for tests. Mention both; pick based on the product.

Complexity

- Time: $O(n)$ swaps, one pass. $n = 52$, effectively constant, but say $O(n)$.
- Space: $O(1)$ extra. In-place.
- `random.shuffle` in CPython is the same algorithm -- say so if asked to reuse `stdlib`.

7. Dealing cards

`draw_card / deal_one`

Pop from the tail. If the deck is empty, raise `EmptyDeckError` -- a dedicated exception, not a bare `ValueError`, so the UI can distinguish 'bad N' from 'deck dry'.

`deal(n)`

Slice the last n cards, delete them, reverse so the first returned card is the old top. This matches n calls to `deal_one` without n resizes of a tiny list.

`deal_hands(players, cards_per_hand) -- fair deal`

Round-robin: for each of N cards, walk every player once. Casino and home-game convention. Alternatives and why we rejected them:

- Give player 1 all N cards, then player 2: faster to slice, but if the shuffle had any residual clump, one player

Deck of 52 Cards

eats a whole run. Also feels unfair at a table.

- Randomly assign remaining cards: more code, same fairness if shuffle is already unbiased.

We pre-check players * N <= remaining. Fail before mutating any hand. That is transactional thinking -- say the word 'all-or-nothing'.

deal_all

Keep dealing round-robin until empty. Leftover cards go to earlier seats (standard leftover convention). Hand sizes differ by at most 1.

8. OOP principles in this design

Encapsulation

Deck._cards, Card._rank/_suit, Player._hand are private. Callers use methods. Player.hand returns a copy so encapsulation is not a suggestion.

Abstraction

The console never builds Card(rank, suit) in a loop. It asks Deck to shuffle and deal. The 'how' of Fisher-Yates is hidden behind shuffle().

Composition over inheritance

Card has-a Suit and Rank. Deck has-a list of Cards. We did not make SpadeCard subclasses. If the interviewer asks 'where is inheritance?', the honest answer is: this domain does not need it. Forcing it would violate YAGNI.

Immutability as a design choice

Cards never change identity. Only the Deck's order and the Player's hand change. That eliminates a class of bugs (a card in two hands that later 'changes suit').

9. SOLID principles

S -- Single Responsibility

Topic	What to say / what we did
Suit / Rank	Vocabulary only. No shuffle, no I/O.
Card	Identity and comparison of one card.
Deck	Pack lifecycle: build, shuffle, draw, reset.
Player	Hold a hand. Does not know Fisher-Yates.
cards_main.py	I/O and menu. If we add poker scoring, it does not go here.

O -- Open/Closed

Adding a joker: introduce Rank.JOKER or a Joker card type and teach Deck to optionally include it. Existing shuffle and deal methods do not change. Adding a new game (poker hand evaluator) is a new module that reads Card.rank/suit -- Deck stays closed.

Honest gap: Ace-high vs Ace-low is baked into Card.__lt__. A Strategy (comparator injected into sort) would be

more OCP if we supported multiple games.

L -- Liskov Substitution

No inheritance hierarchy of cards, so LSP is vacuously true. If we later add `Joker(Card)`, it must still hash/compare without breaking `set(Deck)`. A joker with no suit must not crash `__lt__`.

I -- Interface Segregation

Python has no formal interface here, but the public surface is small: `shuffle`, `draw_card`, `deal_hands`, `reset_deck`. Peek is display-only. We do not force `Player` to implement `shuffle`.

D -- Dependency Inversion

`cards_main` depends on the `Deck` abstraction (public methods), not on the list layout. `Deck` depends on `Card`, not on the console. To unit-test `shuffle`, inject a tiny `Deck([c1,c2,c3])` -- the constructor already accepts an iterable (test seam).

10. Design patterns

Topic	What to say / what we did
Value Object	Card: equality by value, hashable, immutable.
Aggregate / Facade	Deck is the only entry for pack operations. UI talks to Deck, not to the raw list.
Factory (lightweight)	Deck. <code>__init__</code> and <code>reset()</code> manufacture the 52-card product.
Iterator	<code>__iter__</code> / <code>__len__</code> / <code>__bool__</code> make Deck a collection.
Decorator (stdlib)	@total_ordering synthesizes the missing comparisons.
Null/empty object? No	We raise <code>EmptyDeckError</code> instead of returning a dummy card. Prefer fail-fast in interviews unless they ask for Optional.

Patterns we deliberately did not use: Singleton (a global deck is hostile to tests), Observer (no UI subscribers in the domain), Strategy for shuffle (one correct algorithm). Name the unused patterns -- it shows taste, not just a checklist.

11. Complexity cheat sheet

Topic	What to say / what we did
Build / reset	$O(52) = O(1)$ for a standard deck, $O(n)$ generally.
shuffle	$O(n)$ time, $O(1)$ extra space.
draw_card	$O(1)$.
deal(k)	$O(k)$.
deal_hands(P, N)	$O(P*N)$ draws, plus an $O(1)$ capacity check.
sorted hand	$O(h \log h)$ using Card. <code>__lt__</code> .

12. Trade-offs and extensions

- Multiple decks / shoe: `Deck(cards=Deck()._cards * 6)` then shuffle. Track depletion and reshuffle threshold (cut card).
- Seeded shuffle for replay tests: accept an `rng` argument on `shuffle(rng=)`.

- Ace high: pass a key= to sorted(), or a Comparator strategy.
- Thread safety: a lock around shuffle+deal if two tables share a shoe. Usually one deck per table, no lock.
- Persistence: serialize remaining cards as (rank, suit) pairs. Card is already a value object.
- UI: cards_main is a client. A web API would call the same Deck methods.

13. Follow-up questions (with answers)

Why not random.shuffle?

It is Fisher-Yates. Implementing it yourself lets you discuss uniformity. In production I would use random.shuffle or secrets and a comment.

How do you test a shuffle?

Invariant tests: same multiset of cards, length 52, no duplicates. Statistical test: over many trials, P(card C is on top) $\approx$ 1/52. Chi-square if they want rigor. Never assert a specific order after shuffle.

Is your shuffle cryptographically secure?

The algorithm is unbiased given a uniform index. Security is a property of the RNG. secrets.randbelow is the right choice for gambling; random is fine for solitaire.

How would you deal so player 1 is not advantaged?

If the shuffle is unbiased, first card is as random as any other. Round-robin is about social fairness and breaking clumps, not about fixing a biased RNG.

Can two threads draw at once?

Not safely. List pop is not atomic across check-and-pop. Either give each table its own Deck or lock the draw path.

Model a discard pile?

Second list on Deck, or a DiscardPile object. reset() can be 'shuffle discard back into deck' -- that is a new method, not a change to Card.

14. Whiteboard script (10 minutes)

1. Minute 1: requirements + Ace-low assumption + no jokers.
2. Minute 2-3: draw Suit, Rank, Card, Deck, Player. Say composition.
3. Minute 4-5: list as the store, top at the end, O(1) draw.
4. Minute 6-7: write Fisher-Yates. Call out the naive n^n bug.
5. Minute 8: deal_hands round-robin + all-or-nothing check.
6. Minute 9: SOLID in one breath (SRP of Deck vs Card, OCP for jokers).
7. Minute 10: extensions -- shoe, seeded RNG, Ace-high comparator.

15. Common mistakes to avoid

- Using inheritance for suits (RedCard / BlackCard) with no behavior difference.
- Shuffle via sort(key=random) -- biased and $O(n \log n)$.

Deck of 52 Cards

- Returning the same list from `Player.hand` (leaks mutability).
- Silent empty draw (return `None`) without saying `Optional`.
- Forgetting that `Enum.value` is reserved in Python.
- Claiming '`O(1)` shuffle' because `n=52`. Say `O(n)`, then '`n` is 52'.